

Problem 7: Own prediction problem

Difficulty: Medium

Every run uses the standard d8 recipe (LR 0.003, linear decay, 1% warmup, batch size 64) and sees the same **614.4M training tokens**, but only **150,000 unique sequences (153.6M tokens)**, repeated for 4 epochs. Width scales with depth (64 x depth). Reference results from this assignment:

Depth	Unique data (1 epoch)	16x repetition (37.5k seqs)
d4	3.290	3.393
d6	3.057	3.367
d9	2.885	3.640

With unique data bigger is better; with 16x repetition the trend is a U-shape (d6 best, d9 worst).

(a) Rank d4, d6 and d9 at 4x repetition, from lowest (best) to highest final validation loss.

- () A. d4 < d6 < d9
- () B. d6 < d4 < d9
- () C. d6 < d9 < d4
- () D. d9 < d6 < d4

(b) How much does 4x repetition cost d9 relative to unique data ($L_{4x} - L_{unique}$)?

- () i. less than +0.02
- () ii. +0.02 to +0.08
- () iii. more than +0.08

Code diff (experiments/a1_basics/p7_own_question.py, 63 lines)

```
--- /dev/null
+++ b/experiments/a1_basics/p7_own_question.py
+import sys
+from model_config import depth_model_config
+from train import TrainConfig, training_run_name
+
+DEPTHS = (9, 6, 4)
+UNIQUE_DATA_LOSS = {4: 3.290, 6: 3.057, 9: 2.885}
+RUNS = [
+    TrainConfig(model_config=depth_model_config(d), num_train_sequences=150_000,
+                num_epochs=4.0, run_name_suffix="p7-repeat4", wandb_tags=("p7-repeat4",))
+    for d in DEPTHS
+]
+
+def resolve(): # prints each run's final val loss and the ranking
+    import wandb
+    from utils import WANDB_ENTITY, WANDB_PROJECT
+    api = wandb.Api(timeout=60)
+    losses = {}
+    for d, config in zip(DEPTHS, RUNS):
+        runs = [r for r in api.runs(f"{WANDB_ENTITY}/{WANDB_PROJECT}",
+                                   filters={"display_name": {"$regex": f"^{training_run_name(config)}"}})
+                if r.state == "finished" and "contaminated" not in (r.tags or [])]
+        losses[d] = max(runs, key=lambda r: r.created_at).summary["val_loss"]
+    for d in sorted(losses):
+        print(f"d{d}: {losses[d]:.4f} (repetition cost {losses[d] - UNIQUE_DATA_LOSS[d]:+.4f})")
+    print("Ranking:", " < ".join(f"d{d}" for d in sorted(losses, key=losses.get)))
+
+if __name__ == "__main__":
+    if sys.argv[1:] == ["resolve"]:
+        resolve()
+    else:
+        from modal_train import launch_training_jobs
+        launch_training_jobs(RUNS, max_parallel_runs=2)
```

Run and resolve

```
uv run python -m experiments.a1_basics.p7_own_question # train the 3 runs
uv run python -m experiments.a1_basics.p7_own_question resolve # print the answer

d4: final val loss 3.3034 (unique data 3.290, repetition cost +0.0134)
d6: final val loss 3.0812 (unique data 3.057, repetition cost +0.0242)
d9: final val loss 2.9327 (unique data 2.885, repetition cost +0.0477)
Ranking (best to worst): d9 < d6 < d4
```

Answer: (a) D, d9 < d6 < d4; (b) ii, +0.048.

Repetition cost depends on epochs *relative to model capacity*. At 16x, d9 (48M parameters) sees only 38M unique tokens and memorizes them, so the ranking flips. At 4x there are ~3 unique tokens per d9 parameter, so bigger still wins, but the cost roughly doubles with each size step (+0.013, +0.024, +0.048): the warning sign that the ranking will flip with more epochs or bigger models. Students who memorized "repetition hurts big models" answer B; students who memorized "bigger is better" underestimate (b).